

INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 17 · Variables, tipos de datos, constantes y entrada y salida

UA2 · Programación estructurada | Módulo CSTI12010



Persona facilitadora

Giovanni Antonio Coto Calderón

Ruta de la sesión



1. Inicio

Repaso y preguntas
diagnósticas

20 min



2. Desarrollo

Exposición · práctica
guiada · práctica
autónoma

175 min



3. Síntesis

Autoevaluación con lista
de cotejo y preguntas
rápidas

20 min



4. Próxima sesión

Operadores, conversiones
de tipo y decisiones

5 min

Recordemos la sesión anterior



Repaso 1

¿Cuáles son las cuatro etapas de gcc?

Recuerde los archivos .i, .s, .o y .exe.



Repaso 2

¿Qué indica un mensaje como falta.c:4:27: error?

Archivo, línea, columna...

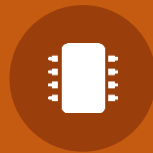
Antes de empezar, piense y responda



Pregunta 1

¿Qué tipo de C cree que corresponde a Entero, Real y Caracter de PSeInt?

Piense en int y en double.



Pregunta 2

¿Cuántos bytes cree que ocupa un número entero en la memoria?

Recuerde la sesión 14: 8 bits = 1 byte.

Capacidad 2.1 y saberes de la sesión

Capacidad procedimental 2.1

Emplear los principales elementos semánticos y sintácticos del lenguaje de desarrollo en el paradigma de programación estructurada.



Del Ser: Autonomía

Valor: Honradez



Del Convivir: Comunicación asertiva

Valor: Respeto

Saberes



1. Variables y tipos



2. Constantes



3. printf y conversiones



4. Modificadores de formato



5. scanf



6. Cadenas

Variables en C



Una **variable** se declara con su **tipo** y su **nombre** antes de usarse: **double precio;**. Puede recibir un valor inicial en la misma línea: **int cantidad = 3;**. La asignación usa **=** (en PSeInt era **<-**).



Tipo primero

`int edad;`



Nombre descriptivo

`camelCase: precioUnitario.`



Distingue mayúsculas

`total ≠ Total.`



Asignación con =

`area = base * altura;`

De PSeInt a C: tipos de datos

PSeInt	C	Ejemplo	printf	scanf
Entero	int	<code>int edad = 20;</code>	<code>%d</code>	<code>%d</code>
Real	double	<code>double precio = 3500.5;</code>	<code>%f</code>	<code>%lf</code>
Caracter (letra)	char	<code>char inicial = 'A';</code>	<code>%c</code>	<code>%c</code>
Caracter (texto)	char[n]	<code>char nombre[30];</code>	<code>%s</code>	<code>%29s</code>
Logico	int (1 / 0)	<code>int esMayor = 1;</code>	<code>%d</code>	<code>%d</code>

Ojo: **double** se muestra con **%f** pero se lee con **%lf**. Una letra va entre comillas simples **'A'** y un texto entre dobles **"Ana"**.

¿Cuánto ocupa cada tipo?

Conexión con la sesión 14: el tamaño en bytes define el **rango** de valores. Salida real con gcc:

tipos.c

```
#include <stdio.h>

int main(void) {
    // sizeof indica cuántos bytes ocupa cada tipo (1 byte = 8 bits)
    printf("char:   %d byte\n", (int) sizeof(char));
    printf("int:    %d bytes\n", (int) sizeof(int));
    printf("float:  %d bytes\n", (int) sizeof(float));
    printf("double: %d bytes\n", (int) sizeof(double));
    return 0;
}
```

Salida real (gcc)

```
char:   1 byte
int:    4 bytes
float:  4 bytes
double: 8 bytes
```

Constantes: #define y const

#define crea una constante **simbólica** (el preprocesador la reemplaza). **const** crea una variable que **no puede cambiar**. Por convención, sus nombres van en **MAYÚSCULAS**.

● ● ● #define (preprocesador)

```
#define PI 3.14159265358979
#define BONO 5000.0

// sin = y sin punto y coma
area = PI * radio * radio;
```

● ● ● const (variable de solo lectura)

```
const double TASA_IVA = 0.13;
const char UNIDAD[] = "cm";

// TASA_IVA = 0.15; <- error
iva = subtotal * TASA_IVA;
```

Constantes: #define y const – Cuándo Usarlas

La diferencia principal es que **#define** es una directiva del preprocesador que reemplaza texto antes de compilar, mientras que **const** es un calificador de tipo que crea una variable real de solo lectura en memoria

● ● ● #define (preprocesador)

```
// Usando #define

#define LIMITE 10

// El preprocesador cambiará cada
palabra "LIMITE" por un 10
```

● ● ● const (variable de solo lectura)

```
// Usando const const

int limite = 10;

// Es una variable entera real, pero no
se puede modificar
```

En el estándar moderno de **C** (como C11, C17 o el más reciente C23), **const** es la opción más recomendada para definir valores constantes. ¿Por qué?

Seguridad de tipos: Si defines **const float PI = 3.14;**, el compilador se asegurará de que se use como un número flotante y te avisará si cometes un error de tipo. Con **#define**, puedes introducir errores sutiles difíciles de detectar.

printf: caracteres de conversión

conversiones.c

```
#include <stdio.h>

int main(void) {
    int edad = 20;
    double precio = 3500.5;
    char inicial = 'A';
    char nombre[] = "Ana";
    printf("%%d    -> %d\n", edad);
    printf("%%f    -> %f\n", precio);
    printf("%%.2f  -> %.2f\n", precio);
    printf("%%c    -> %c\n", inicial);
    printf("%%s    -> %s\n", nombre);
    printf("%%%%    -> %%\n");
    return 0;
}
```

Salida real (gcc)

```
%d    -> 20
%f    -> 3500.500000
%.2f  -> 3500.50
%c    -> A
%s    -> Ana
%%    -> %
```

Nota: En la función printf, el **doble símbolo** %% se utiliza para imprimir un único carácter de porcentaje (%) literal en la pantalla. % es un carácter especial que C utiliza para identificar especificadores de formato (como %d para enteros o %f para flotantes)

Modificadores: ancho, alineación y ceros

Entre **%** y la letra se indica el **ancho**, los **decimales** y la **alineación**. Útil para facturas y comprobantes con columnas.

● ● ● modificadores.c

```
#include <stdio.h>

int main(void) {
    printf("[%8.2f]\n", 3500.5);    // ancho 8, 2 decimales
    printf("[%8s]\n", "Ana");      // izquierda
    printf("[%8s]\n", "Ana");      // derecha
    printf("[%05d]\n", 42);        // ceros a la izquierda
    return 0;
}
```

● ● ● Salida real (gcc)

```
[ 3500.50]
[Ana      ]
[        Ana]
[00042]
```

scanf: leer datos del teclado

Terminal: area.c con 5 y 3

```
Digite la base del rectangulo (cm): 5
Digite la altura del rectangulo (cm): 3
El area del rectangulo es 15.00 cm2
```



printf("Digite ... ");

Mensaje sin `\n`: se escribe en la misma línea.



scanf("%lf", &base);

Lee un double y lo guarda en base.



& (dirección)

Indica dónde guardar el valor.



Leer ↔ scanf

Escribir ↔ printf.

Errores frecuentes con scanf

Estos errores **compilan** (solo dan advertencias con -Wall), pero el programa falla o da valores absurdos.
Lea las advertencias.



Incorrecto

● ● ● Advertencias reales de gcc

```
scanf("%d", edad);  
// warning: format '%d' expects  
//   argument of type 'int *'  
  
double precio;  
scanf("%d", &precio);  
// warning: ... 'int *', but argument  
//   2 has type 'double *'
```



Correcto

● ● ● Formato y & correctos

```
scanf("%d", &edad);  
// & entrega la dirección de edad  
  
double precio;  
scanf("%lf", &precio);  
// %lf para double L minúscula de "Long Float"
```

Caracteres y cadenas



char: un carácter

Comillas simples

- `char inicial = 'A';`
- Ocupa 1 byte (su código: `'A' = 65`).
- Se muestra con `%c`.
- Se lee con `" %c"` (el espacio salta el Enter anterior).



Cadena: arreglo de char

Comillas dobles

- `char nombre[30] = "Ana";`
- Termina con el carácter nulo `'\0'`: "Ana" ocupa 4 casillas.
- Se muestra con `%s` y se lee con `%29s` (sin `&`).
- `%s` lee una sola palabra: el espacio la corta.

De PSeInt a C: AreaRectangulo

El mismo algoritmo de la **sesión 2**, ahora en C. Cambia la escritura; la lógica es idéntica.

AreaRectangulo.psc (UA1)

```
Algoritmo AreaRectangulo
    Definir base, altura, area Como Real;
    Escribir "Digite la base del rectángulo (cm):";
    Leer base;
    Escribir "Digite la altura del rectángulo (cm):";
    Leer altura;
    area <- base * altura;
    Escribir "El área del rectángulo es ", area, " cm2";
FinAlgoritmo
```

area.c (UA2)

```
#include <stdio.h>
int main(void) {
    double base, altura, area;
    printf("Digite la base del rectangulo (cm): ");
    scanf("%lf", &base);
    printf("Digite la altura del rectangulo (cm): ");
    scanf("%lf", &altura);
    area = base * altura;
    printf("El area del rectangulo es %.2f cm2\n", area);
    return 0;
}
```

Recuento de caracteres de salida

printf devuelve un número: la **cantidad de caracteres** que escribió (incluido el **\n**).

● ● ● cuenta.c

```
#include <stdio.h>

int main(void) {
    // printf devuelve cuántos caracteres escribió
    int n = printf("Hola, INA\n");
    printf("printf escribió %d caracteres\n", n);
    return 0;
}
```

● ● ● Salida real (gcc)

```
Hola, INA
printf escribió 10 caracteres
```

Variables y E/S de calidad



Inicializar

Acumuladores en 0 antes de usarlos.



%.2f para dinero

Siempre dos decimales.



Nombres descriptivos

precioUnitario, no p.



Mensaje antes de scanf

La persona sabe qué escribir.



Constantes en MAYÚSCULAS

TASA_IVA, PI, BONO.



Compilar con -Wall

Y corregir cada advertencia.

Paso a paso: de PSeInt a C

A

AreaRectangulo → area.c

B

CirculoPI → circulo.c

C

FacturaConIVA → factura.c

D

InscripcionCurso →
inscripcion.c

E

Errores con scanf

Siga cada paso junto con la persona facilitadora y compruebe el resultado antes de avanzar.

Ahora usted: demuestre lo aprendido



Ejercicio 1 · Temperatura

ConversionTemperatura → C.



Ejercicio 2 · Salario neto

SalarioNeto con const.



Ejercicio 3 · Pago semanal

PagoSemanal con #define y columnas.



Ejercicio 4 · Intercambio

Intercambio de variables.

Documento: S17_Practica_autonoma_Estudiantes.docx · Entrega: carpeta UA2/S17/ del portafolio.

¿Qué aprendimos hoy?

1

¿Con qué formato se lee y se muestra un double?

2

¿Por qué scanf necesita & delante de la variable?

3

¿Qué diferencia hay entre #define y const?

4

¿Cuántas casillas ocupa la cadena "Ana"?



Lista de cotejo: mis programas en C

Cada persona revisa su factura.c y marca:

- Tipos correctos (int / double / char[]).
- %lf en scanf y %.2f en printf para dinero.
- & en cada scanf (salvo cadenas).
- Constantes con nombre en MAYÚSCULAS.
- Compila con -Wall sin advertencias.

Sesión 18: Operadores, conversiones de tipo y decisiones



Operadores

División entera, %, ++ y --



Casting

Mezcla de tipos



if ... else

Decisiones en C

Para pensar: en C, ¿cuánto da $7 / 2$ si ambos son int? ¿Y $7.0 / 2$?